PROJECT STATEMENT

1. Project Title

University Payroll Management System

2. Course

Python Essentials

3. Project Type

Command-Line Python Application

4. Problem Statement

Universities need to manage salary information for faculty members and staff. Manual payroll calculations can be time-consuming and may result in calculation mistakes.

The University Payroll Management System is a beginner-friendly Python application that provides a simple terminal-based way to manage employee records and calculate payroll.

The system allows the user to add employees, view employee information, search for employees, record overtime, generate payslips, create a payroll report, and remove employees.

5. Aim of the Project

The main aim of this project is to develop a simple Python-based payroll application that demonstrates fundamental programming concepts while solving a practical university payroll problem.

6. Objectives

The objectives of the project are:

1. Store employee information using Python data structures.
2. Add and manage faculty and staff records.
3. Calculate employee salaries.
4. Calculate overtime payments.
5. Calculate tax deductions.
6. Calculate retirement deductions.
7. Calculate net salary.
8. Generate an employee payslip.
9. Generate a payroll report.
10. Provide input validation and error handling.
11. Make the project executable entirely from a terminal.

7. Scope

The project covers basic payroll operations for university employees.

The system supports:

• Faculty employees
• Staff employees
• Employee IDs
• Employee names
• Departments
• Job roles
• Monthly salaries
• Overtime hours
• Tax calculation
• Retirement deduction
• Net salary calculation
• Employee search
• Employee removal
• Payroll reports

The current version is intended for educational use and keeps employee data in memory while the program is running.

8. Functional Requirements

FR1 - Add Employee

The system shall allow the user to add a new employee by entering:

• Name
• Department
• Job role
• Employee type
• Monthly salary

FR2 - View Employees

The system shall display all employees currently stored in the application.

FR3 - Search Employee

The system shall allow the user to search employees using a name or department.

FR4 - Record Overtime

The system shall allow overtime hours to be added to an existing employee.

FR5 - Calculate Payroll

The system shall calculate basic salary, overtime pay, gross salary, tax, retirement deduction, and net salary.

FR6 - Generate Payslip

The system shall display a detailed payslip for a selected employee.

FR7 - Generate Payroll Report

The system shall calculate and display payroll totals for all employees.

FR8 - Remove Employee

The system shall allow an employee record to be removed using the employee ID.

9. Non-Functional Requirements

Usability

The application shall provide a simple numbered menu that can be operated from a terminal.

Reliability

The program shall handle invalid numeric input and missing employee IDs without terminating unexpectedly.

Maintainability

The program shall separate major operations into functions such as employee management, payroll calculation, reporting, and the main menu.

Portability

The application shall run on systems with Python installed and shall not require a graphical interface.

Performance

The program should respond quickly for the small number of employee records expected in an educational project.

10. Payroll Calculation Rules

The project uses the following calculation rules.

Hourly Rate

Hourly rate is calculated using 160 working hours per month:

Hourly Rate = Monthly Salary / 160

Overtime Pay

Overtime is paid at 1.5 times the hourly rate:

Overtime Pay = Overtime Hours × Hourly Rate × 1.5

Gross Salary

Gross salary is:

Gross Salary = Basic Salary + Overtime Pay

Faculty Tax

Faculty employees are charged a 10% tax:

Faculty Tax = Gross Salary × 0.10

Staff Tax

Staff employees are charged an 8% tax:

Staff Tax = Gross Salary × 0.08

Retirement Deduction

Retirement deduction is 5%:

Retirement = Gross Salary × 0.05

Net Salary

Net salary is:

Net Salary = Gross Salary - Tax - Retirement

11. Input Validation

The application validates:

• Empty employee names
• Empty departments
• Empty job roles

• Invalid employee type selections
• Invalid salary values
• Non-positive salary values
• Invalid employee IDs
• Invalid overtime values
• Negative overtime hours
• Employee IDs that do not exist

12. Technologies Used

• Python 3
• Lists
• Dictionaries
• Functions
• Loops
• Conditional statements
• Exception handling
• Standard input/output

No third-party Python packages are required.

13. System Modules

The single-file beginner implementation contains the following logical modules/functions:

Function	Purpose
add_employee()	Adds a new employee
view_employees()	Displays employee records
find_employee()	Finds an employee by ID
add_overtime()	Records overtime hours
calculate_payroll()	Calculates salary and deductions
generate_payslip()	Displays an individual payslip
search_employee()	Searches by name or department
remove_employee()	Removes an employee
payroll_report()	Displays payroll totals
main()	Controls the application menu

14. Data Structure

The application uses a list called `employees`.

Each employee is represented by a dictionary containing:

• id

• name

• department

• role

• type

• salary

• overtime

15. Expected Output

The application provides:

• Employee records

• Search results

• Overtime confirmation

• Detailed payslips

• Payroll summaries

• Success and error messages

16. Execution

The application is designed to run from the command line.

Run:

```
python university_payroll.py
```

On systems where Python 3 is invoked separately:

```
python3 university_payroll.py
```

17. Limitations

The current beginner version has the following limitations:

1. Employee data is stored only while the program is running.
2. Data is lost when the application exits.
3. There is no user authentication.
4. There is no database.
5. There is no graphical interface.
6. Payroll rules are fixed in the program.

18. Future Enhancements

Future versions can include:

• SQLite database storage

• Login and authentication

• Role-based access

• Attendance management

• Leave management

• Payroll history

• CSV export

• PDF payslip generation

• Configurable tax rates

• Web-based interface

19. Conclusion

The University Payroll Management System demonstrates how Python can be used to solve a practical administrative problem.

The project applies fundamental Python concepts including variables, lists, dictionaries, functions, loops, conditional statements, exception handling, user input, and arithmetic calculations.

The application is intentionally simple so that the complete program can be understood and executed by a beginner from the command line.